# CoreLAB CPU Class Diagram

procedure(Sender: TObject; Event: TCPUEvent) of object

procedure(Sender: TCPU) of object

**core_cpu**

C «type helper» TArchitectureHelper
+ToString(): string
+FromString(Value: string): TArchitecture

C «type helper» TEndiannessHelper
+ToString(): string
+FromString(Value: string): TEndianness

C «type helper» TSemanticVersionHelper
+ToString(): string
+Compare(AOther: TSemanticVersion): Integer

E TArchitecture
arHarvard
arNeumann

E TEndianness
enLittle
enBig

C «record» TSemanticVersion
+Major: Integer
+Minor: Integer
+Patch: Integer

A *TCPU*

#FOnEvent: TCPUEventHandler
#FModname: PChar
#FDescription: PChar
#FVersion: TSemanticVersion
#FAddressWidth: Byte
#FArchitecture: TArchitecture
#FBusType: TBusType
#FDefaultClockFreq: DWord
#FEnabled: Boolean
#FEndianness: TEndianness
#FLastInstruction: TLastInstruction
#FMaxCodeAddress: DWord
#FMaxIOPortAddress: DWord
#FMaxMemAddress: DWord
#FHasSeparateIOBus: Boolean
#FRunning: Boolean
#FHalted: Boolean
#FInterruptEnabled: Boolean
#FIRQPending: Boolean[8]
#FIRQVector: Byte[8]
#FNMIPending: Boolean
#FTotalCycles: QWord
#FTotalInstructions: QWord
#FOnDestroy: TProcessorDestroyEvent
#FRegPtr: array of ^Word

+AddressWidth: Byte {read-only}
+Architecture: TArchitecture {read-only}
+BusType: TBusType {read-only}
+DefaultClockFreq: DWord {read-only}
+TotalCycles: QWord {read-only}
+Description: PChar {read-only}
+Enabled: Boolean {read-write}
+Endianness: TEndianness {read-only}
+Halted: Boolean {read-only}
+HasSeparateIOBus: Boolean {read-only}
+TotalInstructions: QWord {read-only}
+InterruptEnabled: Boolean {read-only}
+MaxCodeAddress: DWord {read-only}
+MaxIOPortAddress: DWord {read-only}
+MaxMemAddress: DWord {read-only}
+Modname: PChar {read-only}
+OnEvent: TCPUEventHandler {read-write}
+Running: Boolean {read-only}
+Version: TSemanticVersion {read-only}
+OnDestroy: TProcessorDestroyEvent {read-write}

#EmitEvent(AEvent: TCPUEvent): void «virtual»
#DoInterrupt(AEvent: TCPUEvent, AMode: Byte = 0): void «virtual»

+ConnectBus(ABus: IInterface): Boolean «abstract, virtual»
+TCPU()
+Destroy()
+CheckInterrupts(): Boolean «virtual»
+GetCurrentInstruction(): TLogRec «abstract, virtual»
+GetCurrentInterrupt(AMode: Byte = 0): TIntLogRec «virtual»
+GetRegister(RegName: PChar): Word «abstract, virtual»
+GetRegisterCount(): Byte «abstract, virtual»
+GetRegisterName(AIndex: Byte): PChar «abstract, virtual»
+GetRegisterSize(AIndex: Byte): Byte «abstract, virtual»
+LoadState(AStream: TStream): Boolean «abstract, virtual»
+SaveState(AStream: TStream): Boolean «abstract, virtual»
+IRQ(AVector: Byte, AMode: Byte = 0): void «virtual»
+NMI(): void «virtual»
+Reset(): void «abstract, virtual»
+Run(): void «virtual»
+SetRegister(RegName: PChar, AValue: Word): void «abstract, virtual»
+Step(): void «abstract, virtual»
+Stop(): void «virtual»

C «type helper» TCPUEventHelper
+ToString(): string
+FromString(Value: string): TCPUEvent

C «procedure type» TCPUEventHandler

C «procedure type» TProcessorDestroyEvent

C «record» TIntLogRec
+Sender: string[32]
+Vector: string[32]
+Status: string[32]
+Flag: string[32]

C «record» TLogRec
+InstCount: Integer
+Address: string[32]
+OpCode: string[32]
+Mnemonic: string[32]
+Cycles: Byte
+TotalCycles: QWord

C «record» TLastInstruction
+Address: Word
+Opcode: Byte
+NumPrefix: Byte
+Prefixes: Word[2]
+NumOperand: Byte
+Operands: Word[2]
+Mnemonic: string[32]
+Cycles: Byte

E TCPUEvent
ceInstructionBoundary
ceInterrupt
ceHalt
ceReset

extends
extends
extends
FVersion
callback
callback
returns
returns
FLastInstruction
extends
FBusType

Abstract base class.

IRQ channels: 0..7.
CheckInterrupts handles NMI first,
then the first enabled pending IRQ
in ascending channel order.
One interrupt is accepted per call.

The constructor initializes execution
and interrupt state, counters, bus type
(btStandard), default clock frequency
(1,000,000 Hz) and version (0.1.0).

**core_bus (external unit)**

E TBusType
btStandard
btMCS4